

บทที่ 2

ทฤษฎี

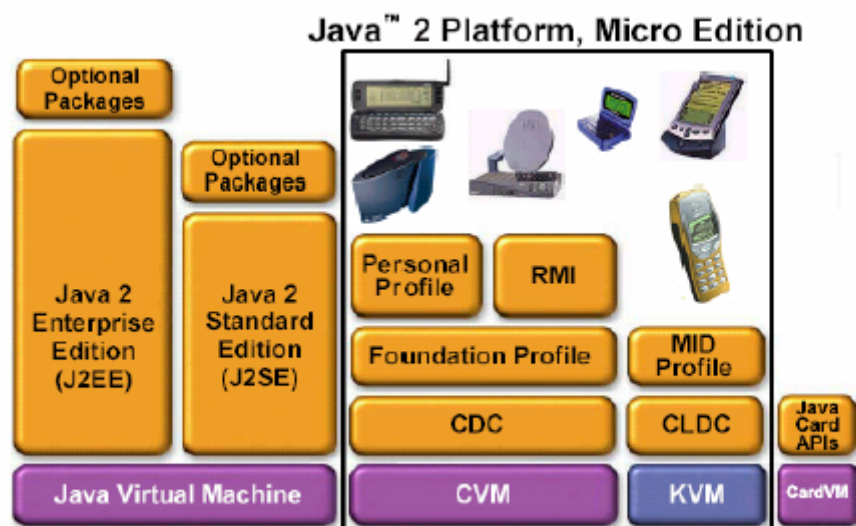
2.1 จาวา

2.1.1 แพลตฟอร์มจาวา

ในช่วงเวลาระยะหลังที่ผ่านมาจาวาได้กลายเป็นแพลตฟอร์ม การพัฒนาแอปพลิเคชันแบบเชิงวัตถุ (object-oriented) ในอุปกรณ์และงานต่างๆ อย่างเต็มตัว นับตั้งแต่แอปพลิเคชันเซิร์ฟเวอร์ระดับองค์กร เครื่องคอมพิวเตอร์แบบตั้งโต๊ะทั่วไป จนถึงแอปพลิเคชันฝังตัวสำหรับอุปกรณ์ขนาดเล็ก

แพลตฟอร์มจาวา[1] ที่ใช้งานอยู่ในขณะนี้ มี 3 รุ่นด้วยกัน แต่ละรุ่นเน้นนำไปใช้กับเฉพาะกลุ่มแอปพลิเคชัน ดังนี้

- Java 2 Enterprise Edition (J2EE) ใช้งานกับแอปพลิเคชันบนเครื่องเซิร์ฟเวอร์สำหรับองค์กรที่รองรับระบบงานใหญ่ๆ และไคลเอนต์จำนวนมาก
- Java 2 Standard Edition (J2SE) ใช้งานกับแอปพลิเคชันบนเครื่องคอมพิวเตอร์แบบตั้งโต๊ะทั่วไป
- Java 2 Micro Edition (J2ME) ใช้งานกับแอปพลิเคชันรุ่นใหม่ซึ่งเน้นกลุ่มผู้ใช้อุปกรณ์อิเล็กทรอนิกส์และอุปกรณ์ฝังตัว (Embedded)



รูปที่ 2.1 แสดงแพลตฟอร์มจาวา 2 ทั้ง 3 รุ่นที่ใช้งานอยู่ในขณะนี้

จาวาแต่ละรุ่นจะมีเวอร์ชวลแมชีน (Virtual Machine) เฉพาะซึ่งปรับแต่งเป็นพิเศษ เพื่อให้สนับสนุนการทำงานของแอปพลิเคชันที่จะนำไปใช้ได้ดียิ่งขึ้น HotSpot VM เป็น Virtual

Machine ของจาวาที่ปรับแต่งเพื่อเพิ่มประสิทธิภาพของแอปพลิเคชันบนเครื่องเซิร์ฟเวอร์ที่เขียนด้วย J2EE

JVM เป็น Virtual Machine สำหรับจาวาทั่วไปที่ปรับแต่งเพื่อให้งานกับแอปพลิเคชันบนเครื่องคอมพิวเตอร์แบบตั้งโต๊ะ นอกจากนี้ HotSpot VM ยังสามารถทำงานเข้ากับแอปพลิเคชันที่เขียนด้วย J2SE ได้เช่นกัน

Virtual Machine ที่ออกแบบมาเพื่อให้ใช้งานสำหรับ J2ME มี 2 ประเภทด้วยกัน คือ C Virtual Machine (CVM) และ K Virtual Machine (KVM) ซึ่งมีขนาดกะทัดรัดและใช้ทรัพยากรของระบบทรัพยากรน้อยกว่า JVM

2.1.2 J2ME

J2ME เป็นแพลตฟอร์มจาวาที่ออกแบบมาเพื่อใช้งานกับแอปพลิเคชันที่ทำงานบนอุปกรณ์ขนาดเล็ก เช่น โทรศัพท์เคลื่อนที่ พีดีเอ โทรศัพท์พร้อมจอภาพที่เชื่อมต่อกับอินเทอร์เน็ต โทรศัพท์ดิจิทัลขนาดเล็ก อุปกรณ์บันทึกและระบบนำทางในรถยนต์ สวิตช์ในระบบเครือข่ายคอมพิวเตอร์ของเครื่องอำนวยความสะดวกภายในบ้าน ฯลฯ

J2ME ได้นำโครงสร้างแบบโมดูลที่มีความยืดหยุ่นสูงเข้ามาใช้ เพื่อให้สามารถสนับสนุนการทำงานอุปกรณ์หลากหลายประเภท จากรูปที่ 2.1 แสดงให้เห็นว่า J2ME กำหนดชั้นการทำงานของซอฟต์แวร์ไว้ 3 ชั้น(Layer)ด้วยกัน โดยเลเยอร์ทั้งหมดจะอยู่เหนือชั้นระบบปฏิบัติการของอุปกรณ์ ดังนี้

- ชั้น Java Virtual Machine เป็นเลเยอร์ของ Java Virtual Machine ปรับแต่งให้เข้ากับระบบปฏิบัติการของอุปกรณ์ และรองรับแต่ละ configuration ของ J2ME จากรูปที่ 2.1 Virtual Machine ของ J2ME ได้แก่ CVM และ KVM
- ชั้น Configuration เป็นเลเยอร์ของคอนฟิกูเรชันของ J2ME ซึ่งกำหนดคลาสไลบรารีอุปกรณ์ใช้งานทั่วไปหรือกลุ่มอุปกรณ์ที่มีความต้องการหน่วยความจำและ หน่วยประมวลผลใกล้เคียงกัน จากรูปที่ 2.1 Configuration ใน J2ME มี 2 ประเภทด้วยกัน คือ CDC (Connected Device Configuration) และ CLDC (Connected Limited Device Configuration)
- ชั้น Profile เป็นเลเยอร์ที่สร้างเหนือเลเยอร์คอนฟิกูเรชัน โดยกำหนดคลาสไลบรารีเพื่อสนองตอบความต้องการของตลาดเฉพาะกลุ่ม จากรูปที่ 2.1 ตัวอย่างของ Profile ชั้นนี้ ได้แก่ PDAP, MIDP, Foundation Profile และ Personal Profile

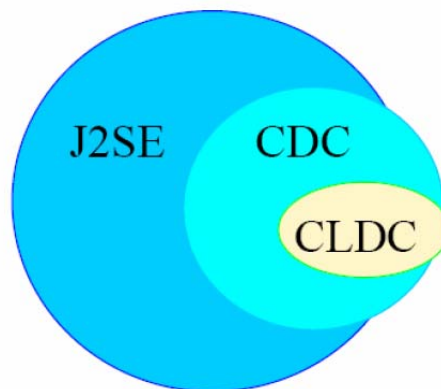
คอนฟิกูเรชันใน J2ME

คอนฟิกูเรชันและโพรไฟล์เป็นองค์ประกอบหลักของ J2ME โดยมีจุดสำคัญ คือ เพื่อปรับแต่ง Virtual Machine และคลาสไลบรารีให้เหมาะสมกับอุปกรณ์แต่ละประเภท คอนฟิกูเรชันคือ ชุดคำสั่งที่มีคุณสมบัติขั้นต่ำของ Java Virtual Machine และจาวาคลาสสำหรับอุปกรณ์แต่ละ

ประเภท เป็นตัวแทนของอุปกรณ์ที่ลักษณะคล้ายคลึงกัน อาจกล่าวได้ว่า คอนฟิกูเรชันเป็นตัวกำหนดคุณสมบัติหรือไลบรารีกำหนดคอนฟิกูเรชันจะมีในอุปกรณ์ที่อยู่ในกลุ่มเดียวกัน

ปัจจุบัน คอนฟิกูเรชันใน J2ME แบ่งเป็น 2 ประเภท คือ CDC (Connected Device Configuration) และ CLDC (Connected Limited Device Configuration) โดยมีเป้าหมายที่กลุ่มอุปกรณ์ 2 ประเภทซึ่งมีความต้องการหน่วยความจำและหน่วยประมวลผลพื้นฐานใกล้เคียงกัน

- CDC เป็นอุปกรณ์ใช้ร่วมกับอุปกรณ์อื่น ติดตั้งตายตัว และใช้เชื่อมต่อข้อมูล โดยปกติมักมีส่วนติดต่อกับผู้ใช้หลากหลายแบบ มีหน่วยความจำประมาณ 2-16 เมกกะไบต์ ใช้หน่วยประมวลผลแบบ 32 บิต หรือมากกว่า เชื่อมต่อเครือข่ายที่มีแบนด์วิดธ์สูงอย่างต่อเนื่อง โดยอาศัยพอร์ต TCP/IP ตัวอย่างอุปกรณ์ประเภทนี้ ได้แก่ โทรศัพท์มือถือขนาดเล็ก อินเทอร์เน็ตทีวี โทรศัพท์พร้อมจอภาพที่เชื่อมต่อกับอินเทอร์เน็ต อุปกรณ์สื่อสารที่มีความซับซ้อนสูง อุปกรณ์บันทึกและระบบนำทางในรถยนต์
- CLDC เป็นอุปกรณ์ส่วนบุคคล พกพาได้ และใช้เชื่อมต่อข้อมูล โดยปกติมักมีส่วนติดต่อกับผู้ใช้แบบง่ายๆ(เมื่อเปรียบเทียบกับระบบบนเครื่องคอมพิวเตอร์แบบตั้งโต๊ะ) มีหน่วยความจำประมาณ 128 กิโลไบต์ – 1 เมกกะไบต์ ใช้หน่วยประมวลผลแบบ 16 บิต หรือ 32 บิต เชื่อมต่อกับเครือข่ายที่มีแบนด์วิดธ์ต่ำเป็นระยะเวลานานๆ โดยไม่อาศัยพอร์ต TCP/IP ตัวอย่างอุปกรณ์ประเภทนี้ ได้แก่ โทรศัพท์เคลื่อนที่แบบไม่ซับซ้อนมากนัก เพจเจอร์รับส่งข้อความ เครื่องปาล์มโอเอสแบบพกพา



รูปที่ 2.2 แสดงความสัมพันธ์ระหว่าง J2SE และคลาสไลบรารีใน CDC และ CLDC

ในชั้นของ Configuration มีคลาส 2 ประเภทด้วยกัน คือ คลาสที่นำมาจาก J2SE และที่ออกแบบเฉพาะอุปกรณ์ขนาดเล็ก คลาสที่นำมาจาก J2SE จะมีคุณสมบัติอย่างเดียวกันกับคลาสใน J2SE หรือซับคลาสของ J2SE เช่น แพ็กเกจ java.lang java.io และ java.util จากรูปที่ 2-2 จะแสดงความสัมพันธ์ระหว่าง J2SE และคลาสไลบรารีของ CDC และ CLDC จากภาพจะเห็นว่าคลาสใน CLDC ส่วนใหญ่ทำงานร่วมกับคลาส CDC ได้ดีเช่นเดียวกับกรณีของคลาสใน CLDC

ขณะเดียวกันคลาสที่ไม่ได้นำมาจาก J2SE และออกแบบเพื่อใช้เฉพาะอุปกรณ์มักทำงานร่วมกับ J2SE ได้ไม่ค่อยดี ใน CLDC คลาสเหล่านี้จะอยู่ในกลุ่มกรอบการติดต่อสื่อสารทั่วไป (Generic Connection Framework) โดยระบุไว้ในแพ็คเกจ javax.microedition.io

J2ME สำหรับอุปกรณ์ไร้สาย

J2ME ได้ให้กำเนิดแอปพลิเคชันยุคใหม่บนอุปกรณ์ไร้สาย ช่วยให้เกมแบบหลายผู้เล่นที่ทำงานผ่านอินเทอร์เน็ต การทำธุรกรรมทางการเงินบนโทรศัพท์เคลื่อนที่ แอปพลิเคชันสำหรับองค์กรทั้งไคลเอ็นท์และเซิร์ฟเวอร์ เกิดขึ้นได้บนโทรศัพท์เคลื่อนที่ และเพจเจอร์รับส่งข้อความ MIDP CLDC และ KVM ได้กลายมาเป็นรากฐานในการพัฒนาแอปพลิเคชันบนอุปกรณ์ไร้สายยุคใหม่

สามารถแบ่งโครงสร้างออกเป็นชั้น(Layer)ต่างๆ จากล่างขึ้นบน ดังนี้

- ชั้น ฮาร์ดแวร์ MID หมายถึง ตัวโทรศัพท์เคลื่อนที่ (เช่น Nokia6600) หรือเพจเจอร์รับส่งข้อความ (เช่น RIM รุ่น Blackberry950)
- ชั้น ซอฟต์แวร์ของระบบที่ติดตั้งมากับอุปกรณ์ หมายถึง ระบบปฏิบัติการและไลบรารีของระบบที่บริษัทผู้ผลิตให้มา
- ชั้น KVM เป็นส่วนที่เตรียม runtime environment ไว้ให้แอปพลิเคชันบนอุปกรณ์ไร้สาย
- ชั้น CLDC เป็นส่วนที่เตรียม API หลักของจาวาให้แอปพลิเคชันบนอุปกรณ์ไร้สาย
- ชั้น MIDP (Mobile Information Device Profile) เป็นส่วนที่เตรียมไลบรารีสำหรับ ส่วนติดต่อกราฟิกกับผู้ใช้ (User Interface) และหน่วยเก็บข้อมูล persistent storage ระบบเครือข่าย และไทม์เมอร์

นอกจากคลาสไลบรารีสำหรับ MIDP อาจเตรียมคลาสไลบรารีเฉพาะอุปกรณ์ไว้ให้นักพัฒนาเพื่อดึงความสามารถของฟังก์ชันที่มีอยู่แล้วไปใช้งานให้เกิดประโยชน์สูงสุด อาทิ การโทรศัพท์ การแชร์ข้อมูลกับแอปพลิเคชันที่ติดตั้งมาในเครื่อง (เช่น ปฏิทิน สมุดจดที่อยู่) การตรวจสอบข้อมูลอุปกรณ์ที่ผู้ผลิตเตรียมไว้มาใช้งาน จะช่วยเพิ่มความสามารถแก่แอปพลิเคชันบนอุปกรณ์ไร้สาย แต่ไม่สามารถเคลื่อนย้ายไปสู่อุปกรณ์อื่นที่ใช้ MIDP ได้ เนื่องจากคลาสที่นำมาใช้อยู่นอกเหนือขอบเขตของ MIDP

ความต้องการของระบบ

อุปกรณ์ไร้สายจะทำงานสนับสนุน J2ME ได้ดีเมื่อมีคุณสมบัติตรงตามข้อกำหนดหากต้องการให้ KVM ทำงานได้อย่างมีประสิทธิภาพ โดยใช้ไลบรารี CLDC จะต้องมีความสัมพันธ์ของระบบขั้นต่ำ ดังนี้

- มีหน่วยความจำขนาด 160 – 512 กิโลไบต์ สำหรับสร้างแพลตฟอร์มจาวา
- มีหน่วยประมวลผลแบบ 16 – 32 บิต ความเร็ว 25 เมกกะเฮิร์ตซ์
- ใช้พลังงานน้อย โดยมากมักทำงานโดยใช้แบตเตอรี่

- เชื่อมต่อกับเครือข่ายได้ในช่วงสั้นๆ (อาศัยระบบไร้สายเป็นส่วนใหญ่) แบนด์วิดท์
- ค่อนข้างจำกัด (ความเร็ว 9600 ไบต์ต่อวินาทีหรือน้อยกว่า)
- มีหน่วยความจำชั่วคราวขนาด 32 กิโลไบต์ สำหรับเก็บจาวา รันไทม์และหน่วยความจำของออบเจกต์

2.1.3 Mobile Media API (MMAPI)

MMAPI[5] เป็นแพ็คเกจ(package) JSR 135 สร้างขึ้นใหม่เพิ่มเติมให้ J2ME มีเครื่องมือต่าง ๆ มากมาย ทำงานร่วมกับ MIDP 2.0 ซึ่งมีคลาส อินเตอร์เฟส ค่าคงที่และเมธอดต่างๆ เพื่อช่วยอำนวยความสะดวกในการสร้างแอปพลิเคชันไร้สายสำหรับเสียง(audio) วิดีโอ (video) ต่างๆ ได้ง่ายขึ้น

MMAPI มีคุณลักษณะต่างๆ ดังนี้

- API ใช้บนอุปกรณ์ของ CLDC ที่มีหน่วยความจำและความเร็วจำกัด
- API ไม่ยึดติดกับโปรโตคอลและประเภทเนื้อหาใดโดยเฉพาะ
- API ออกแบบเพื่อให้สามารถเพิ่มคุณลักษณะใหม่ๆ ได้ในภายหลัง

การเชื่อมโยงโปรโตคอลและเนื้อหาต่างๆ ของโปรโตคอล (Protocol and Contents Handling)

การดำเนินการมัลติมีเดีย จะมี 2 ขั้นตอนดังนี้

- การเชื่อมโยงกับโปรโตคอลของการจัดส่งข้อมูล
- การเชื่อมโยงกับเนื้อหาของข้อมูล

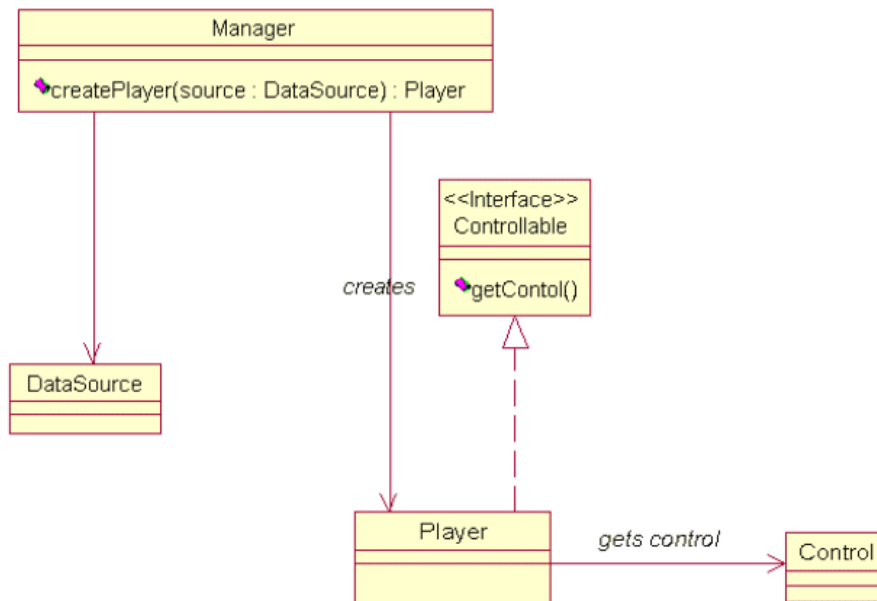
การเชื่อมโยงกับโปรโตคอล

การเชื่อมโยงกับโปรโตคอลจะอ้างถึงการอ่านข้อมูลจากแหล่งต่างๆ (เช่น ไฟล์อุปกรณ์จับภาพหรือเซิร์ฟเวอร์จัดการบรรจุและจัดส่งข้อมูล) ให้ระบบดำเนินการสื่อ

การเชื่อมโยงเนื้อหา

ปกติจะดำเนินการข้อมูลของสื่อ (เช่น การจำแนกหรือลงรหัส เป็นต้น) และนำสื่อไปยังอุปกรณ์เอาพุต เช่น ลำโพงหรือเครื่องเล่นวิดีโอ

API ของสื่อ (เสียงและวิดีโอ) ที่ใช้กับโทรศัพท์เคลื่อนที่นั้น จะมีคลาสที่สำคัญ ได้แก่ DataSource และ Manager ช่วยดำเนินการมัลติมีเดียตามที่แสดงในรูป



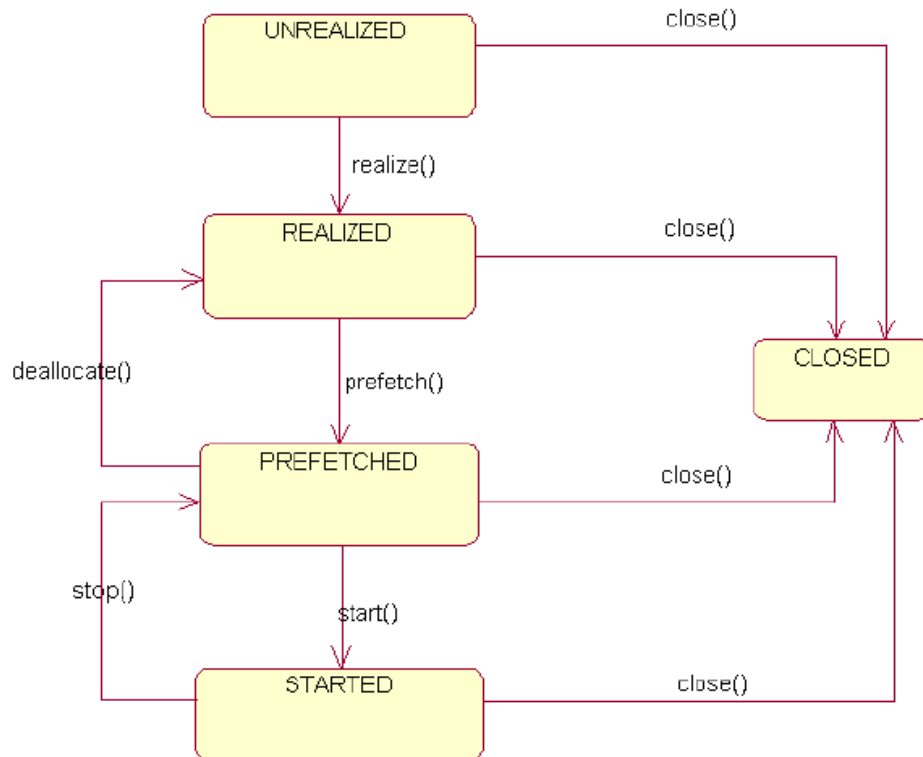
รูปที่ 2.3 แสดงการทำงานร่วมกันระหว่างคลาส DataSource และ Player

- คลาส DataSource ให้การสนับสนุนการเชื่อมโยงโปรโตคอล
- อินเทอร์เฟซ Player ให้การสนับสนุนการเชื่อมโยงเนื้อหา
- คลาส Manager ช่วยสร้างออบเจกต์ของคลาส Player อ่านข้อมูลของออบเจกต์ของคลาส DataSource

คลาส DataSource จะทำการซ่อนรายละเอียดต่างๆของโปรโตคอลเกี่ยวกับวิธีการอ่านข้อมูลจากแหล่งข้อมูลของมันไม่ว่าข้อมูลจะมาจากไฟล์ เซิร์ฟเวอร์สำหรับจัดส่งข้อมูล กลไกการจัดส่งข้อมูล DataSource จะมีเมธอดต่างๆช่วยให้คลาส Player สามารถอ่านข้อมูลจากมันเข้าไปประมวลผล

คลาส Player อ่านข้อมูลจากคลาส Datasource และทำการประมวลผลเสร็จแล้วส่งไปให้อุปกรณ์เอาต์พุต ซึ่งมีมีเมธอดต่างๆช่วยควบคุมการเล่นเสียงและวิดีโอ รวมทั้งการแลกเปลี่ยนข้อมูลพื้นฐาน นอกจากนี้ยังช่วยควบคุมการเข้าถึงคุณลักษณะต่างๆสำหรับสื่อประเภทต่างๆที่ต้องการ

เมื่อ Player ถูกสร้างจะอยู่ใน Unrealized state เรียกเมธอด realize() ให้ Player ทำการติดต่อไปยังที่อยู่ของ resource เมื่อเมธอดส่งค่ากลับมาจะอยู่ใน realized state เรียกเมธอด prefetch() ทำให้ Player รอรับ resource เพื่อให้ในการเล่นไฟล์ ก่อนเข้าสู่ prefetch state ถึงจุดนี้ Player สามารถเล่นไฟล์ได้แล้ว โดยการเรียกเมธอด start() ซึ่งวัฏจักรของคลาส Player แสดงได้ดังรูปที่ 2.4



รูปที่ 2.4 แสดงวัฏจักรการทำงานของคลาส Player

คลาส Manager มีเมธอดต่างๆทำหน้าที่เสมือนเป็นโรงงานสร้างออปเจ็ทของคลาส Player (จาก URL ที่ระบุโปรโตคอลและเนื้อหา) และสร้างออปเจ็ทของคลาส DataSource เมธอด createPlayer() ของคลาส Manager ช่วยสร้างออปเจ็ทของคลาส Player ดังนี้

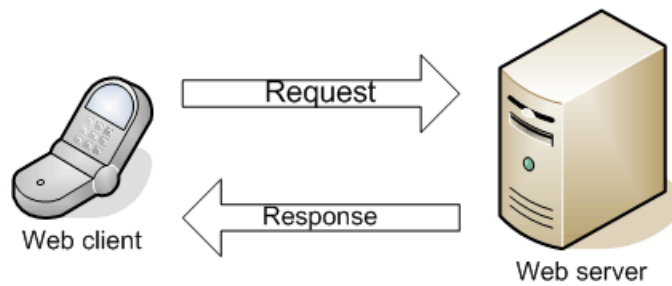
Player play=Manager.createPlayer(String url);

- url ระบุโปรโตคอลและเนื้อหาของข้อมูลในรูปแบบ <protocol>:<content location>
เช่น ใช้ “capture://video” เป็น url ระบุโปรโตคอล capture และที่อยู่ของเนื้อหาวิดีโอ

คลาส Manager จะทำการจำแนก URL และสร้างออปเจ็ทของคลาส DataSource เชื่อมโยงกับโปรโตคอลที่จัดส่งข้อมูลโดยเฉพาะ และสร้างออปเจ็ทของ Player อ่านเนื้อหาของออปเจ็ทของคลาส DataSource ไปแสดงที่อุปกรณ์แสดงผล

2.1.4 HTTP Protocol

HTTP เป็นโปรโตคอล(protocol) ช่วยกำหนดแบบสื่อสาร ไคลเอนต์ที่เป็นเว็บเบราว์เซอร์ จะทำการร้องขอทรัพยากรไปยังเซิร์ฟเวอร์ และเซิร์ฟเวอร์จะส่งสิ่งต่างๆตอบสนองกับไปยังเว็บเบราว์เซอร์ HTTP เป็นโปรโตคอลไม่มีสถานะ หมายความว่า เซิร์ฟเวอร์ไม่จำเป็นต้องดูแลสารสนเทศใดๆภายหลังที่มันส่งการตอบสนองไปยังไคลเอนต์ และไม่จำเป็นต้องจำแนกการร้องขอต่างๆจากไคลเอนต์



รูปที่ 2.5 แสดงการร้องขอและการตอบสนองทรัพยากรของ HTTP Protocol

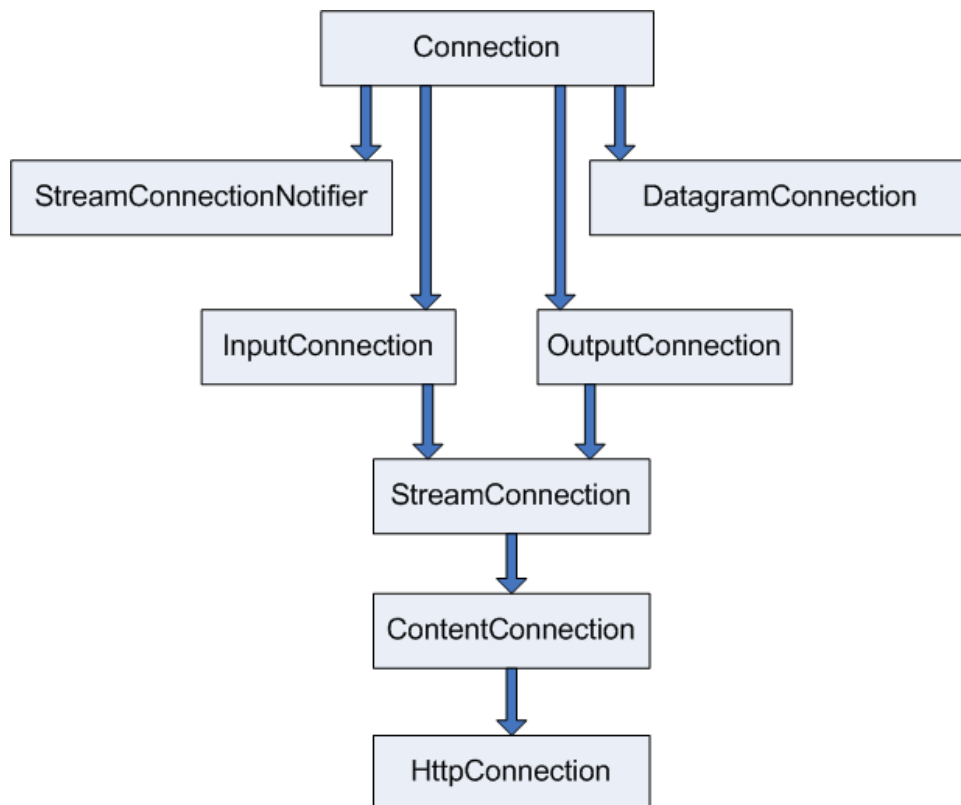
คลาสและอินเตอร์เฟส ของการเชื่อมโยง (The Generic Connection Class and Interfaces)

คลาส (class) และอินเตอร์เฟส (interface) ต่างๆของการเชื่อมโยงทั่วไปที่บรรจุใน CLDC ของ J2ME จะให้การสนับสนุนการเชื่อมโยงของเครือข่ายขนาดเล็กและ I/O ของไฟล์สำหรับความเร็วในการส่งข้อมูลผ่านอุปกรณ์เคลื่อนที่ต่างๆคลาสและอินเตอร์เฟสต่างๆของเครือข่ายและ I/O ของไฟล์ของ J2SE จากแพ็คเกจ java.io และ java.net จะมีขนาดใหญ่กว่าคลาสและอินเตอร์เฟสต่างๆของ J2ME ที่รวมอยู่ในแพ็คเกจ javax.microedition.io

คลาสและอินเตอร์เฟสต่างๆเหล่านี้มีสมรรถนะสูงและมีความยืดหยุ่นในการสนับสนุนการเชื่อมโยงของเครือข่ายสำหรับอุปกรณ์ไร้สาย

อินเตอร์เฟส Connection (Connection Interface)

อินเตอร์เฟส Connection เป็นอินเตอร์เฟสพื้นฐาน อินเตอร์เฟสอื่นๆทั้งหมดสืบทอดจากคลาส Connection นี้ อินเตอร์เฟส StreamConnection สืบทอดจากคลาส InputConnection และ OutputConnection มันจะให้การสนับสนุนอินพุตและเอาต์พุตสำหรับการเชื่อมโยงของสตรีม (stream) อินเตอร์เฟส ContentConnection จะสืบทอดคุณลักษณะจากอินเตอร์เฟส StreamConnection มันได้เพิ่มสามเมธอดสำหรับการจัดการข้อมูลของ MIME เชื่อมโยงกับเมธอดสำหรับอินพุตและเอาต์พุตในอินเตอร์เฟส StreamConnection ทำให้อินเตอร์เฟส HttpURLConnection สืบทอดจากอินเตอร์เฟส ContentConnection

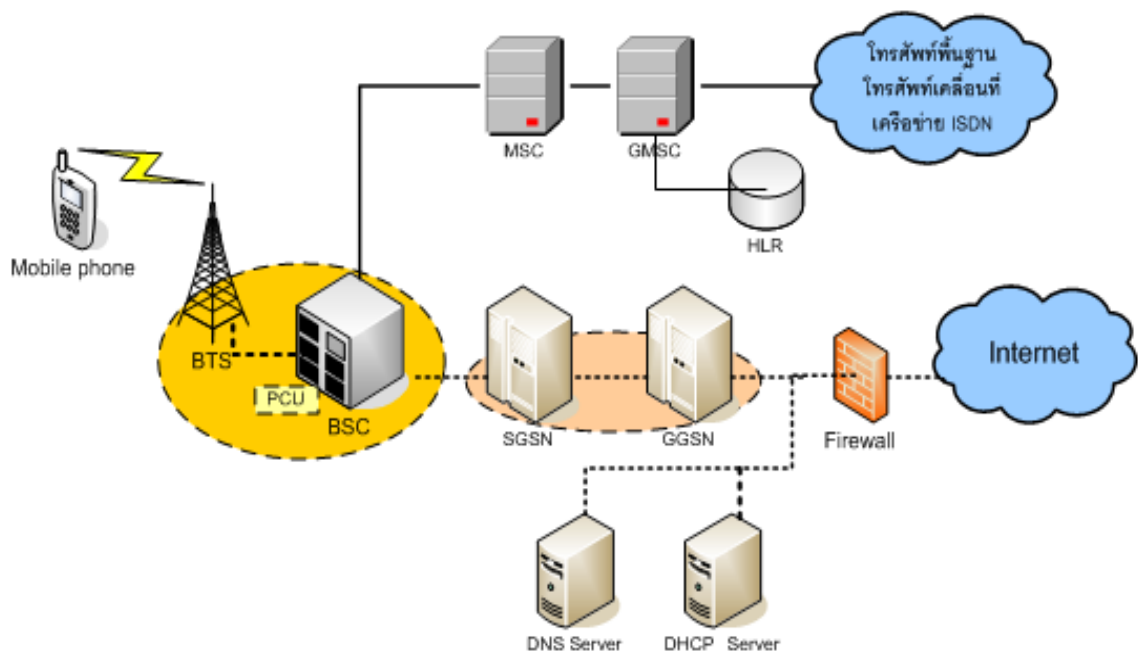


รูปที่ 2.6 แสดงการสืบทอดอินเตอร์เฟสต่างๆของคลาส Connection

HttpConnection ไม่ใช่ส่วนหนึ่งของคลาสและอินเตอร์เฟสของการเชื่อมโยงทั่วไป มันได้กำหนดไว้ใน MIDP เพื่อใช้งานกับโทรศัพท์เคลื่อนที่ สองทาง HttpConnection บรรจุเมธอดและค่าคงที่ต่างๆเพื่อสนับสนุนโปรโตคอล HTTP 1.1

2.2 GPRS

แนวคิดของการวางเครือข่ายโทรศัพท์เคลื่อนที่ GPRS[7] เกิดขึ้นบนพื้นฐานของการใช้โครงสร้างที่มีอยู่เดิมของเครือข่ายโทรศัพท์เคลื่อนที่ GSM ในการเพิ่มขยายขีดความสามารถในการรับส่งข้อมูลในรูปแบบแพ็กเก็ต ซึ่งเป็นเทคโนโลยีการสื่อสารที่ไม่มีอยู่บนเครือข่ายโทรศัพท์ GSM จึงจำเป็นที่จะต้องมีการติดตั้งอุปกรณ์เครือข่ายเพิ่มเติม ทั้งที่ทำหน้าที่เป็นอุปกรณ์ชุมสายโทรศัพท์เคลื่อนที่แบบสวิตช์แพ็กเก็ต และการปรับต่างขีดความสามารถของเครือข่ายสถานีฐาน (Base Station Subsystem หรือ BSS) ให้สามารถรับส่งข้อมูลแบบสวิตช์แพ็กเก็ตผ่านคลื่นความถี่วิทยุ เพื่อติดต่อไปยังเครื่องลูกข่ายโทรศัพท์เคลื่อนที่ GPRS ได้อย่างถูกต้อง



รูปที่ 2.7 แสดงโครงสร้างของเครือข่าย GPRS

อุปกรณ์ที่จำเป็นต้องมีการติดตั้งเพิ่มเติม มีรายละเอียดและหน้าที่การทำงานดังนี้

- Serving GPRS Support Node หรือ SGSN ทำหน้าที่ ในลักษณะเดียวกับ MSC ซึ่งเป็นชุมสายโทรศัพท์เคลื่อนที่ของเครือข่าย GSM ต่างกันที่อุปกรณ์ MSC มีรูปแบบการเชื่อมต่อวงจรสื่อสารแบบสวิตช์วงจร ในขณะที่อุปกรณ์ SGSN มีการส่งข้อมูลในรูปแบบแพ็กเก็ต
- Gateway GPRS Support Node หรือ GGSN ทำหน้าที่ คล้ายคลึงกับชุมสายเกตเวย์ (Gateway MSC หรือ GMSC) ซึ่งเป็นปากทางในการเชื่อมต่อวงจรสื่อสารบนเครือข่ายโทรศัพท์เคลื่อนที่ GSM กับเครือข่ายโทรศัพท์ภายนอก โดยอุปกรณ์ GGSN จะเป็นปากทางในการติดต่อสื่อสารกับเครือข่ายคอมพิวเตอร์ภายนอก ไม่ว่าจะเป็นเครือข่ายเฉพาะองค์กร (Private Data Network) หรือ เครือข่ายสาธารณะ เช่น อินเทอร์เน็ต
- Packet Control Unit หรือ PCU เป็นอุปกรณ์เสริมที่ได้รับการออกแบบสร้างขึ้นสำหรับเพิ่มขีดความสามารถให้กับเครือข่ายสถานีฐานของเครือข่าย GSM ที่มีอยู่เดิมในการแยกข้อมูลที่อยู่ในรูปของการสื่อสารแบบแพ็กเก็ตไปยังอุปกรณ์ SGSN สำหรับข้อมูลการสนทนาที่อยู่ในรูปแบบของการเชื่อมต่อแบบสวิตช์วงจรก็ยังคงมีการรับส่งโดยตรงระหว่างเครือข่ายสถานีฐานกับอุปกรณ์ MSC ตามปกติ
- GPRS Register หรือ GR เป็นส่วนขยายซึ่งอยู่ในรูปแบบของระบบฐานข้อมูล ที่จะต้องได้รับการขยายเพิ่มเติมขึ้นในอุปกรณ์ HLR ของเครือข่ายโทรศัพท์เคลื่อนที่ GSM โดย GR เป็นฐานข้อมูลที่เก็บรวบรวมข้อมูลเกี่ยวกับผู้ใช้บริการโทรศัพท์เคลื่อนที่ GPRS ซึ่งเป็นที่แน่นอนว่าจะต้องเป็นผู้ใช้บริการที่มีการลงทะเบียนไว้ในอุปกรณ์ HLR ในฐานะ

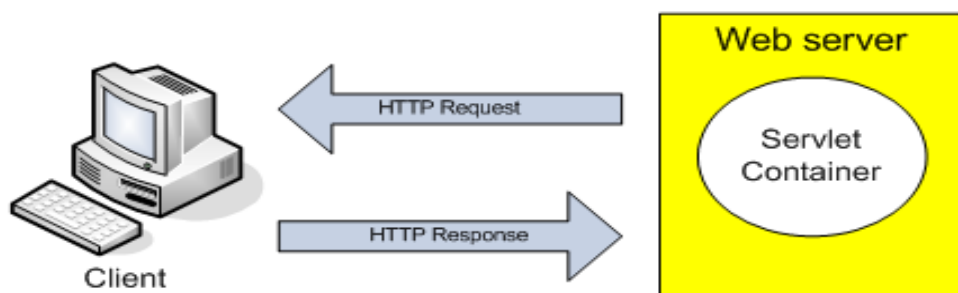
ของผู้ให้บริการ GSM การเพิ่มเติมส่วนขยายของฐานข้อมูล HLR ซึ่งถือเป็นการเพิ่มเติมในด้านของซอฟต์แวร์จึงถือเป็นแนวทางในการพัฒนาเครือข่าย GSM ไปเป็นเครือข่าย GPRS ที่เหมาะสมที่สุด

- Domain Name Server หรือ DNS เป็นอุปกรณ์ซึ่งมีลักษณะเป็นเครื่องคอมพิวเตอร์เซิร์ฟเวอร์ ทำหน้าที่ในการเก็บรวบรวมฐานข้อมูลแสดงความสัมพันธ์ระหว่างรายชื่อของเว็บไซต์หรือแอดเดรสของเครื่องคอมพิวเตอร์ในรูปแบบ URL ทั้งที่อยู่ในเครือข่าย GPRS หรือ ในเครือข่ายภายนอกกับเลขหมายแอดเดรส IP เป็นอุปกรณ์มาตรฐานที่มีการติดตั้งใช้งานอยู่ทั่วไปภายในเครือข่ายอินเทอร์เน็ต
- Dynamic Host Control Protocol Server หรือ DHCP Server เป็นอุปกรณ์คอมพิวเตอร์เซิร์ฟเวอร์ ใช้สำหรับการกำหนดจัดสรรเลขหมาย IP ให้กับผู้ใช้บริการ GPRS แบบชั่วคราว
- Firewall เป็นอุปกรณ์มาตรฐานที่มีการติดตั้งใช้งานในเครือข่ายคอมพิวเตอร์ทั่วไป ทำหน้าที่ควบคุมดูแลและป้องกันการรับส่งข้อมูลในรูปแบบแพ็กเก็ต หรือ IP ผ่านกฎเกณฑ์ที่มีการกำหนดไว้ภายในตัวอุปกรณ์ Firewall ตัวอย่างเช่น ไม่ยินยอมให้มีการส่งข้อมูลออกไปยัง IP ที่ไม่พึงประสงค์ หรือ ไม่รับข้อมูลจากเครือข่ายภายนอกที่มีการส่งจาก IP หมายเลขใดหมายเลขหนึ่ง ถือเป็นการป้องกันการคุกคามการทำงานของเครือข่าย GPRS ที่มีประสิทธิภาพมาก

2.3 Servlet

Servlet คือ Component ที่พัฒนาขึ้นด้วย Java Technology ให้มีความสามารถในการจัดการกับ Dynamic Content ของ Web Application ซึ่ง Servlets นั้นจะเป็นโปรแกรมที่ทำงานอยู่บน Web Server ทำหน้าที่เป็นตัวกลางระหว่างคำสั่งที่ได้รับจาก Web Browser หรือ HTTP Client กับฐานข้อมูล หรือ Application ต่างๆบน HTTP Server

Servlet จะมี Servlet Container หรือ เรียกว่า “Servlet Engines” ซึ่งเป็นส่วนหนึ่งของ Web Server เป็นตัวคอยจัดการการโต้ตอบระหว่าง Web Client กับ Servlets ที่อยู่ทาง Web Server



รูปที่ 2.8 แสดงรูปแบบการทำงานของ Servlet

Servlets ได้รับการออกแบบให้ทำงานภายใต้ Request/Response Model ภายใน Request/Response Model นั้น Client จะส่ง Request Message ไปที่ Server และ Server จะตอบกลับด้วย Reply Message โดยที่ Request สามารถอยู่ในรูปแบบของ HTTP URL, FTP URL หรือ Custom Protocol

Request และ Response แสดงให้เห็นถึงสถานะของ client และ Server ในขณะที่มีการรับส่งข้อมูลระหว่างกัน โดยทั่วไปแล้ว สถานะของ Client/Server Connection จะไม่ถูกเก็บไว้ภายใต้ Request/Response ในช่วงเวลาที่แตกต่างกัน แต่หากเก็บค่าของ Request/Response เหล่านั้นไว้ในภายใน Session แล้ว ข้อมูลจะได้รับการเก็บไว้ใน Servlets

2.3.1 Java Servlet API

API หรือ Application Programming Interface เป็นรูปแบบมาตรฐานที่ผู้พัฒนาใช้อ้างอิงในการพัฒนา Application ซึ่ง API จะเกี่ยวข้องกับการเข้าถึงลักษณะ และสถานะของ Classes และ Objects ต่างๆที่มีในโปรแกรม

Java Application Programming Interface (Java API) เป็นมาตรฐาน JDK (java Development Kit) ที่ผู้พัฒนาใช้อ้างอิงในการพัฒนา Java Program โดยจะถูกเก็บรวบรวมไว้ภายใต้ Package ของ java เช่น java.io.* หรือ java.util.* เป็นต้น สำหรับ Java Servlet API นั้นเป็นส่วนที่เพิ่มเติมขึ้นจากมาตรฐาน JDK หรือที่เรียกว่า JDK Extension โดย Interface ต่างๆจะถูกเก็บรวบรวมไว้ภายใต้ Package ของ javax

2.3.2 โครงสร้างของ Servlet

Servlet Interface เป็นองค์ประกอบที่สำคัญของ Servlet API ทุก Servlets จะ implement interface นี้โดยตรง หรือโดยการ extend Class ที่ implement interface นี้ Class ที่ implement Servlet interface ได้แก่ GenericServlet และ HttpServlet โดยส่วนใหญ่ผู้พัฒนาจะสามารถใช้ HttpServlet ในการพัฒนา Servlet

Servlet interface มี service Method สำหรับจัดการกับ Client Requests ซึ่ง Method นี้จะถูกเรียกใช้สำหรับทุกๆ Request ที่ Servlet Container (ซึ่งมีหน้าที่จัดการกับ Request และ Response) จะจัดการกับ Instance ของ Servlet

HttpServlet Abstract Subclass มี Method อื่นๆนอกเหนือจากที่มีใน Servlet interface ซึ่ง Method เหล่านั้นจะถูกเรียกโดย service Method ใน HttpServlet class เพื่อจัดการกับการประมวลผล HTTP Requests เช่น doGet สำหรับจัดการกับ HTTP GET Requests doPost สำหรับจัดการกับ HTTP POST Requests

2.2.3 วัฏจักรของ Servlets

Servlets จะ run อยู่บน Web Server ภายใต้ Process เดียวกับ Web Server ดังนั้น Web Server จะเป็นผู้รับผิดชอบการเริ่มต้น (initializing), การทำงาน (Invoking) และการทำลาย (Destroy) แต่ละ Instance ของ Servlet องค์ประกอบที่สำคัญในวัฏจักรของ Servlet คือ Servlet

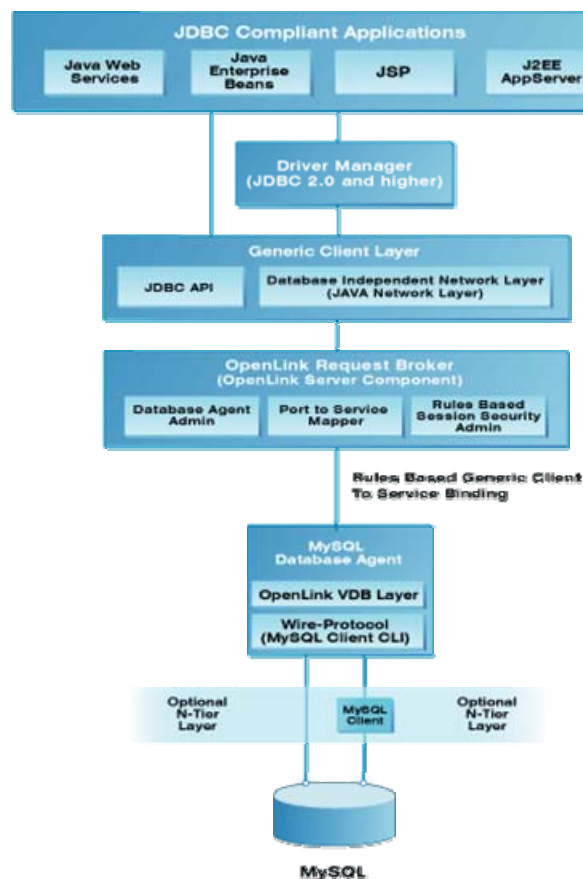
Container ที่มีหน้าที่จัดการกับ Request และ Response โดย Servlet Container จะต้องสนับสนุนการทำงานของ HTTP Protocol ซึ่งเป็น Protocol สำหรับ Request และ Response โดย Web Server สื่อสารกับ Servlet ผ่านทาง Interface ที่เรียกว่า “javax.servlet.Servlet”

2.4 JDBC

JDBC เปรียบเสมือนตัวกลางที่เชื่อมโยงระหว่าง Java Program และ Database โดย JDBC มี Standard Library สำหรับการเข้าถึง Database การใช้ JDBC API (JDBC Application Programming Interface) จะทำให้ Application สามารถเข้าถึง Database ดึงข้อมูล หรือเปลี่ยนแปลงข้อมูลใน Database ที่หลากหลายด้วย Java Syntax เดียวกัน

JDBC ถูกสร้างขึ้นโดยมีชุดของ Java Interfaces ที่เรียกว่า “Drivers” เป็นตัวแยกแยะว่า Database ตัวนั้นจะพูดคุยกับ JDBC ได้อย่างไร Database แต่ละชนิดที่สนับสนุนการใช้งาน JDBC จะมีชุดของ Drivers สำหรับ Database นั้นๆ ความสามารถที่โดดเด่นของ JDBC คือ การที่ทำให้ผู้พัฒนาไม่จำเป็นต้องทราบกลไกที่ซับซ้อนภายในแต่ละ Database ในการพัฒนา Application ทั้งนี้ JDBC จะซ่อนกลไกที่ซับซ้อนเฉพาะแต่ละ Database ไว้ แต่จะบอกให้ผู้พัฒนาทราบถึง interface ที่ใช้ในการถึง Database ที่สนับสนุนการใช้งาน JDBC

JDBC ใช้ interface ที่ง่ายต่อการเข้าใจ JDBC จะถูกนำมาใช้งานร่วมกับ Structured Query Language (SQL) SQL เป็นภาษาที่เป็นที่นิยมโดยทั่วไปสำหรับการจัดการกับข้อมูลใน Database จากการที่ JDBC ใช้มาตรฐานของภาษา SQL ทำให้ JDBC สามารถสนับสนุนการใช้งาน Database ได้เป็นจำนวนมาก



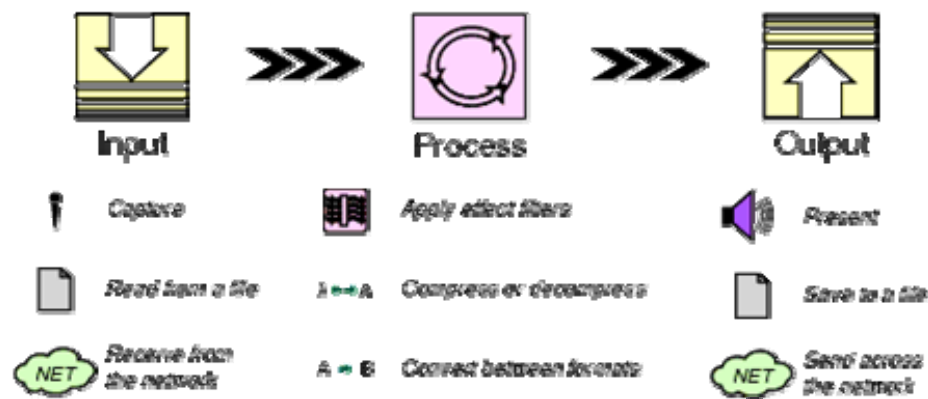
รูปที่ 2.9 แสดงสถาปัตยกรรม MySQL Multi-Tier JDBC Driver

2.5 Java Media Framework

ในภาษาจาวาได้มีการพัฒนา API ขึ้นมาเพื่อจัดการเกี่ยวกับเรื่องของข้อมูลมัลติมีเดียซึ่งมีเอฟีโอที่พัฒนาขึ้นนั้นคือ Java Media Framework (JMF)[2],[9] ซึ่งเป็นเอฟีโอในการจัดการเกี่ยวกับข้อมูลมัลติมีเดีย (Audio และ Video) ซึ่ง Java Media Framework เอฟีโอ สนับสนุนการทำงานกับ Time-based media และ Real time media ดังนั้นหากมีการนำ JMF ไปใช้งานร่วมกับการเขียนโปรแกรมด้วยภาษาจาวาก็จะสามารถเพิ่มความสามารถในการพัฒนาโปรแกรมส่วนการรับส่งข้อมูลมัลติมีเดียให้มีประสิทธิภาพยิ่งขึ้น

2.5.1 Time-base media

Time-base media คือการที่มีข้อมูลที่มีเปลี่ยนแปลงข้อมูลที่มีการเกี่ยวข้องหรือขึ้นอยู่กับเวลาจะเรียกเป็น Time-base media โดยมีตัวอย่างเช่น Video clips, MIDI Sequence, Movie Clips โดยข้อมูลมัลติมีเดียต่างก็มีแหล่งกำเนิดข้อมูลมาจากหลายที่เช่น Local หรือ network File, cameras, microphones, live broadcasts เป็นต้น ในรูปที่2. แสดง Model ของ Media Process Data ของข้อมูลแบบ Time-base media



รูปที่ 2.10 แสดงโมเดลโปรเซสของข้อมูลแบบ Time-base media

ใน Time-base media นั้นจะต้องการเวลาและโปรเซสต่างๆเป็นตัวแทน โดย Timebase-media นั้นจะอยู่ในรูปแบบที่เรียกว่า media stream ซึ่ง media stream ส่วนมากจะประกอบด้วยหลายๆ Channel ของข้อมูลซึ่งเรียกว่า Track ตัวอย่างเช่น ในรูปแบบไฟล์ QuickTime จะประกอบด้วย 2 track คือ audio track และ video track โดย Media stream ที่ประกอบด้วยหลายๆ tracks คือ Multiplex track หลายๆ track เข้าด้วยกัน ซึ่งจะมีรูปแบบ (Format) ของ track ที่กำหนดว่า ข้อมูลของ track นั้นมีโครงสร้างข้อมูลเป็นอย่างไร

รูปแบบทั่วไปของข้อมูลมัลติมีเดียต่างๆมีรายละเอียดดังตารางที่ 2.1 และตารางที่ 2.2 ซึ่งในตารางได้นำเสนอรูปแบบต่างๆและบ่งบอกถึงรูปแบบของมีเดีย คุณภาพของมีเดีย การใช้งาน ซีพียู และ การใช้งานของแบนด์วิดท์

ตารางที่ 2.1 แสดงรายละเอียดของรูปแบบมีเดียของวิดีโอต่างๆ

รูปแบบ	Context Type	คุณภาพ	ความต้องการของ CPU	ความต้องการของแบนด์วิดท์
MPEG-1	MPEG	High	High	High
H.261	AVI RTP	Low	Medium	Medium
H.263	QuickTime AVI RTP	Medium	Medium	Low
JPEG	QuickTime AVI RTP	High	High	High
Indeo	QuickTime AVI	Medium	Medium	Medium

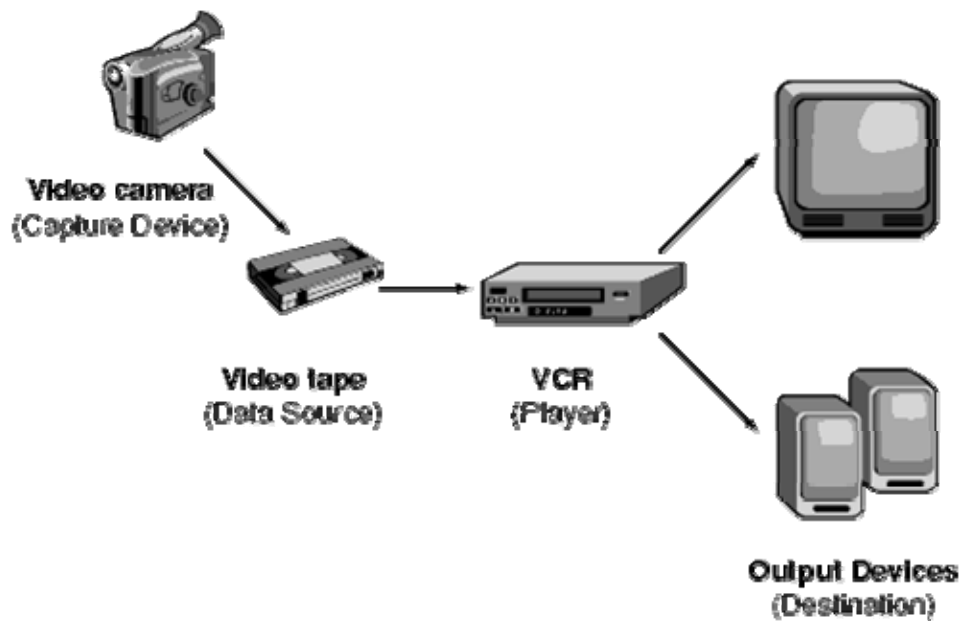
ตารางที่ 2.2 แสดงรายละเอียดของรูปแบบมีเดียของเสียงต่างๆ

รูปแบบ	Context Type	คุณภาพ	ความต้องการของ CPU	ความต้องการของแบนด์วิดท์
PCM	AVI QuickTime WAV	High	Low	High
Mu-Law	AVI QuickTime WAV RTP	Low	Low	High
ADPCM (DVI, IMA4)	AVI QuickTime WAV RTP	Medium	Medium	Medium
MPEG-1	MPEG	High	High	High
MPEG Layer3	MPEG	High	High	Medium
GSM	WAV RTP	Low	Low	Low
G.723.1	WAV RTP	Medium	Medium	Low

โดยรูปแบบมีเดียที่มีความต้องการซีพียูสูงและแบนด์วิดท์สูงนั้นส่วนมากจะอยู่ในรูปแบบของมีเดียที่อยู่ในซีดีรอมหรือStorageใน Local H.232 และ H.263 ส่วนมากจะใช้ในแอปพลิเคชันที่เกี่ยวกับ Video Conference ซึ่งจะเป็นรูปแบบวิดีโอที่ไม่มีการเปลี่ยนแปลงมากนัก G.723.1 จะอยู่ในแอปพลิเคชันใน Telephony ซึ่งต้องการข้อมูลมัลติมีเดียที่มีแบนด์วิดท์ที่ต่ำ

2.5.2 โครงสร้างของ Java Media Framework

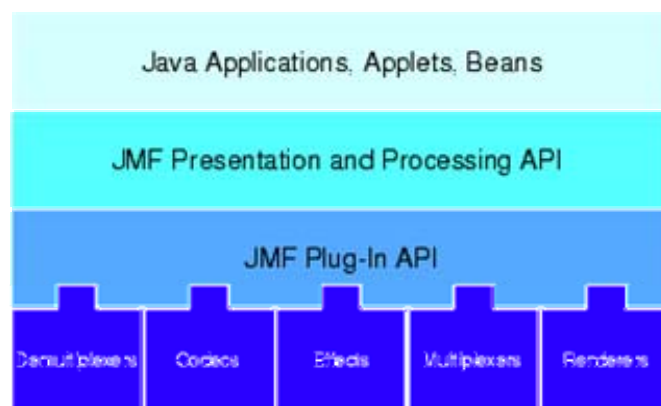
โครงสร้างของ Java Media Framework สามารถเปรียบเทียบได้ดังรูปที่ 2.11 ซึ่งแสดงถึงการทำงานของ การ Recording, Processing และ Presentation ของ Java media Framework



รูปที่ 2.11 แสดงโครงสร้างของ Java Media Framework API

โดย Java Media Framework ได้ใช้โมเดลดังรูปที่ 2.11 เป็นรูปแบบโครงสร้างของการพัฒนาโดยมี Data Source เปรียบได้ดัง วิดีโอเทป และกระบวนการ Player ซึ่งไว้ใช้สำหรับการดำเนินกระบวนการต่างๆเกี่ยวกับมีเดียและการควบคุมมีเดียซึ่งเหมือนกับเครื่องเล่นวิดีโอ ส่วนการที่จะรับภาพเข้ามา JMF จะต้องการ Input จาก Capture Device ที่เปรียบได้ดังกับกล้องวิดีโอ ส่วน Output สามารถเป็นได้ทั้ง ไมโครโฟน,จอภาพ,ลำโพง

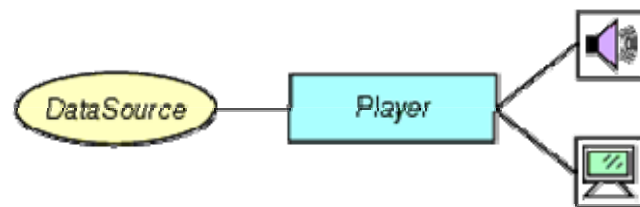
Data Source และ Player นั้นจะรวมอยู่ในส่วน Java Media Framework high-level API สำหรับการจัดการเกี่ยวกับการ Capture และ Process ต่างๆของ time-based media ที่แสดงในรูป 2.1 และJava Media Framework ยังประกอบด้วย low-level API ที่รองรับการทำงานในการปรับปรุงเปลี่ยนแปลง process ที่มีอยู่แล้วให้ได้ตามที่ต้องการ เพื่อให้ผู้ทำการพัฒนาสามารถทำการเปลี่ยนแปลงที่จะรองรับการทำงานของข้อมูลมัลติมีเดียจากเทคโนโลยีใหม่ๆได้ตามต้องการ



รูปที่ 2.12 แสดงโครงสร้างของ High-level Java Media Framework API

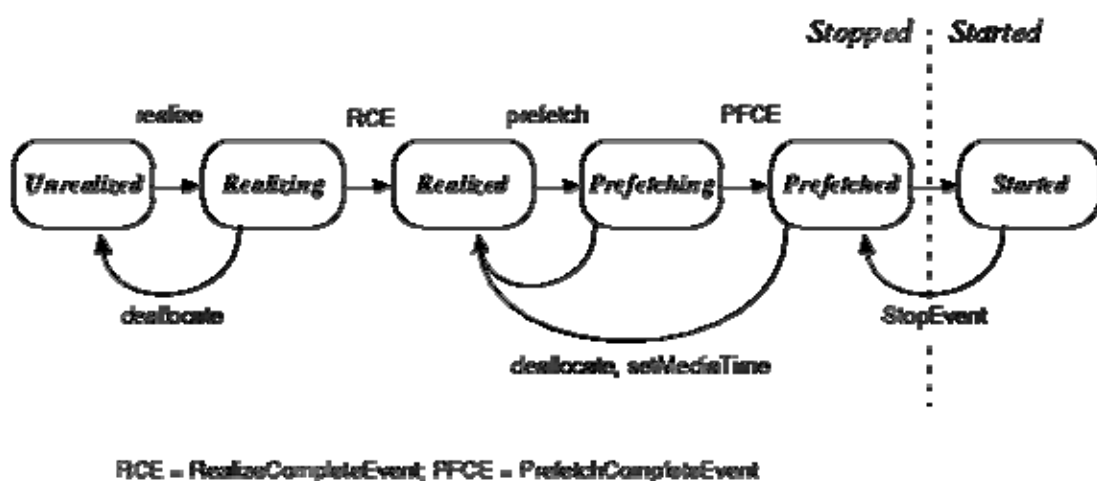
โดยโครงสร้าง High-level Java Media Framework ประกอบด้วยส่วน Presentation ซึ่งเป็นส่วนจัดการเกี่ยวกับการควบคุมกระบวนการ Player และ Capture ข้อมูลมัลติมีเดียต่างๆ ส่วน Processing ซึ่งมีการทำงานคล้ายกับ Presentation แต่มีการทำงานที่ยืดหยุ่นกว่า และส่วน Java media Framework Plug-in ซึ่งประกอบด้วย การทำ Demultiplexer ,Multiplexer ,Encode ,Decode ให้ข้อมูลในรูปแบบต่างๆ เช่น H261, H263, Mpeg1 ส่วน Effect ที่ทำการเปลี่ยนแปลงภาพต่างๆ ให้กับข้อมูลที่รับเข้ามา และส่วน Renderer ใช้สำหรับการแสดงภาพและเสียงไปยังทางจอภาพและลำโพง

โดยในชั้น Presentation นั้นมีส่วนประกอบคือ กระบวนการควบคุมข้อมูลมัลติมีเดียที่รับเข้ามากับ และกระบวนการในการ Player ซึ่งมีกระบวนการทำงานคือการรับ Data Source ที่เตรียมไว้มาทำการแสดงบนอุปกรณ์ที่ต้องการ โดยมีโมเดลแสดงดังรูปที่ 2.13



รูปที่ 2.13 แสดงโมเดลของกระบวนการ Player ในชั้น Presentation

และ Player ได้มีการแบ่ง State การทำงาน โดยแบ่งออกเป็น State ต่างๆ ประกอบไปด้วยทั้งหมด 6 State การทำงานแสดงดังรูปที่ 2.14

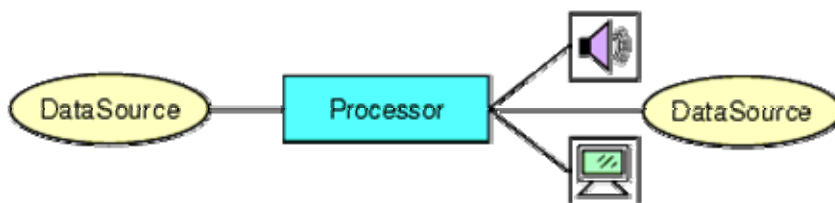


รูปที่ 2. 14 แสดง State ต่างๆของกระบวนการ Player ในชั้น Presentation

กระบวนการ Player ในการทำงานแบบปกติจะทำงานแต่ละ State จนกระทั่งถึง Started State โดยแต่ละ State จะมีการทำงานดังนี้

- Unrealized State การทำงานในUnrealized นั้นจะทำงานโดยใช้เวลาน้อยมากใน State นี้ ซึ่งจะทำให้การสร้างมีเดีย Player ขึ้นมาแต่ยังไม่รู้ว่ามีเดียดังกล่าวมีรูปแบบเป็นอย่างไร
- Realizing State จะเข้าสู่การทำงาน State นี้ เมื่อถูกเรียกให้ทำงาน จะทำงานในการตรวจสอบข้อมูลมีเดียว่ามีอยู่ในรูปแบบอะไร และทำการตรวจสอบว่ากระบวนการ Player สามารถทำการแสดงข้อมูลมีเดียที่ไปยังอุปกรณ์ต่างๆได้หรือไม่
- Realized State เมื่อ Player นั้นเสร็จการ Realizing State จะทำการข้าม State มาที่ Realized ซึ่งใน State นี้จะรู้ว่าอุปกรณ์ใดบ้างที่สามารถทำการแสดงมีเดียนี้ได้และรู้ถึงข้อมูลของมีเดียที่จะแสดง
- Prefetching State จะเข้าสู่การทำงาน State นี้ เมื่อถูกเรียกให้ทำงาน โดยมาจาก Realized State Prefetching มีหน้าที่ทำการเตรียมการนำข้อมูลมีเดียออกแสดงโดยทำการ Preload ข้อมูลมีเดียต่างๆไว้ให้กับอุปกรณ์
- Prefetched State จะเข้าสู่การทำงานใน State นี้เมื่อเสร็จการทำงานใน State Prefetching ซึ่งมีเดียพร้อมที่จะทำการเริ่มแสดงได้
- Stated state จะเข้าสู่การทำงานใน state นี้เมื่อถูกเรียกการทำงาน จะทำการแสดงข้อมูลมีเดียไปยังอุปกรณ์ที่ต้องการ

ส่วนในชั้น Processing นั้นสามารถนำมาใช้ในการแสดงข้อมูลมีเดียคล้ายกับกระบวนการ Player แต่เป็นการกระบวนการ Player ชนิดพิเศษซึ่งสามารถนำ Data Source ที่รับเข้ามาทำการเอาที่พุดออกไปเป็น DataSource อีกตัวหนึ่งได้ซึ่งสามารถนำไปใช้กับกระบวนการ Player ตัวอื่นหรือนำไปใช้ในการบันทึกเป็นไฟล์ได้ โดยมีโมเดลแสดง ดังรูปที่ 2.15



รูปที่ 2.15 แสดงโมเดลในชั้น Processing

2.2.3 รูปแบบมีเดียที่ Java Media Framework รองรับการทำงาน

JAVA Media Framework API ในเวอร์ชันปัจจุบันได้รองรับการทำงานรูปแบบไฟล์ต่างๆ ในแพลตฟอร์มต่างๆ ดังตารางที่ 2.3 ดังนี้

ตัวย่อที่อธิบายความหมายในตารางดังนี้

- D หมายถึง รูปแบบดังกล่าวสามารถนำมา Decoded และ Present ได้

- **E** หมายถึง สามารถ Encoded ในรูปแบบต่างๆ ได้
- **read** หมายถึง สามารถทำการรับรูปแบบนั้นเป็นอินพุตที่ได้(การอ่านจากไฟล์)
- **write** หมายถึง สามารถทำการเขียนรูปแบบดังกล่าวเป็นเอาต์พุตได้(การเขียนไปที่ไฟล์)

ตารางที่ 2.3 แสดงรายละเอียดของรูปแบบมีเดียที่ Java media Framework รองรับ

Media Type	JMF 2.1.1 Cross Platform Version	JMF 2.1.1 Solaris/Linux Performance Pack	JMF 2.1.1 Windows Performance Pack
AIFF (.aiff)	read/write	read/write	read/write
8-bit mono/stereo linear	D,E	D,E	D,E
16-bit mono/stereo linear	D,E	D,E	D,E
G.711 (U-law)	D,E	D,E	D,E
A-law	D	D	D
IMA4 ADPCM	D,E	D,E	D,E
AVI (.avi)	read/write	read/write	read/write
Audio: 8-bit mono/stereo linear	D,E	D,E	D,E
Audio: 16-bit mono/stereo linear	D,E	D,E	D,E
Audio: G.711 (U-law)	D,E	D,E	D,E
Audio: A-law	D	D	D
Audio: GSM mono	D,E	D,E	D,E
Audio: ACM**	-	-	D,E
Video: Cinepak	D	D,E	D
Video: MJPEG (422)	D	D,E	D,E
Video: RGB	D,E	D,E	D,E
Video: YUV	D,E	D,E	D,E
Video: VCM**	-	-	D,E
GSM (.gsm)	read/write	read/write	read/write
GSM mono audio	D,E	D,E	D,E
HotMedia (.mvr)	read only	read only	read only

Media Type	JMF 2.1.1 Cross Platform Version	JMF 2.1.1 Solaris/Linux Performance Pack	JMF 2.1.1 Windows Performance Pack
IBM HotMedia	D	D	D
MIDI (.mid)	read only	read only	read only
Type 1 & 2 MIDI	-	D	D
MPEG-1 Video (.mpg)	-	read only	read only
Multiplexed System stream	-	D	D
Video-only stream	-	D	D
MPEG Layer II Audio (.mp2)	read only	read/write	read/write
MPEG layer 1, 2 audio	D	D,E	D,E
QuickTime (.mov)	read/write	read/write	read/write
Audio: 8 bits mono/stereo linear	D,E	D,E	D,E
Audio: 16 bits mono/stereo linear	D,E	D,E	D,E
Audio: A-law	D	D	D
Audio: GSM mono	D,E	D,E	D,E
Audio: IMA4 ADPCM	D,E	D,E	D,E
Video: Cinepak	D	D,E	D
Video: H.261	-	D	D
Video: H.263	D	D,E	D,E
Video: JPEG (420, 422, 444)	D	D,E	D,E
Video: RGB	D,E	D,E	D,E
Sun Audio (.au)	read/write	read/write	read/write
8 bits mono/stereo linear	D,E	D,E	D,E
16 bits mono/stereo linear	D,E	D,E	D,E

Media Type	JMF 2.1.1	JMF 2.1.1	JMF 2.1.1
	Cross Platform Version	Solaris/Linux Performance Pack	Windows Performance Pack
G.711 (U-law)	D,E	D,E	D,E
A-law	D	D	D
Wave (.wav)	read/write	read/write	read/write
8-bit mono/stereo linear	D,E	D,E	D,E
16-bit mono/stereo linear	D,E	D,E	D,E
G.711 (U-law)	D,E	D,E	D,E
A-law	D	D	D
GSM mono	D,E	D,E	D,E
DVI ADPCM	D,E	D,E	D,E
MS ADPCM	D	D	D

2.6 Java Mail

Java Mail[10] เป็น package ที่อยู่ในเทคโนโลยี J2EE ใช้สำหรับเสริมสำหรับการอ่านเขียนและส่งอีเมลล์สามารถใช้ Package ในการสร้าง Mail User Agent (MUA) ซึ่งเป็นการสร้างโปรแกรมในการส่งอีเมลล์ชนิดหนึ่ง คล้ายกับโปรแกรม Eudora, pine และ Microsoft Outlook ได้รวมถึงมีความสามารถรองรับโปรโตคอล SMTP (Simple Mail Transfer Protocol), pop3(Post Office Protocol), IMAP(Internet Message Access Protocol) เมื่อใช้ Java Mail อีพีไอในการพัฒนาแอปพลิเคชันจะสามารถทำให้แอปพลิเคชันที่ทำการพัฒนาขึ้นมา สามารถทำการส่งอีเมลล์อ่าน อีเมลล์จาก Storage รวมถึงทำการสร้างอีเมลล์ได้

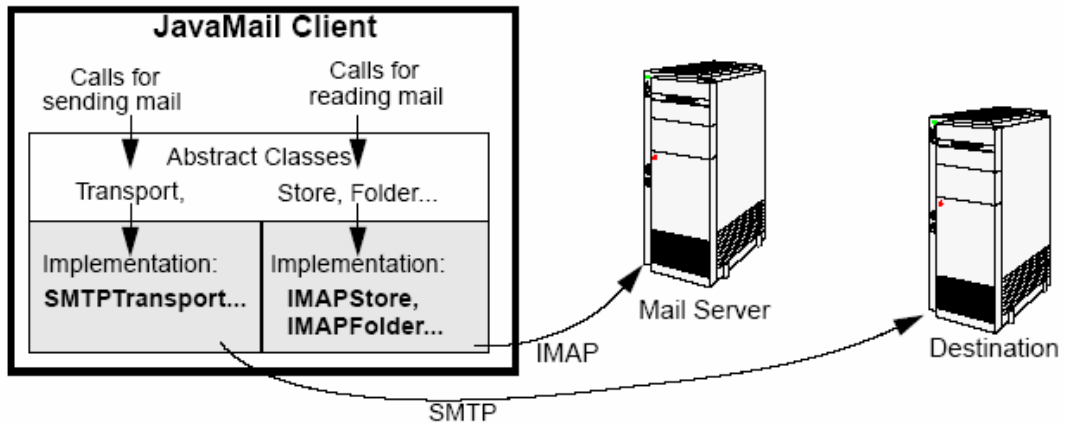
2.6.1 ส่วนประกอบของ Java Mail

Java Mail Api ประกอบด้วย Abstract Class ใช้สำหรับการทำงานในระบบ mail ซึ่งประกอบด้วย Class ต่างๆดังต่อไปนี้

- Message เป็นคลาสที่ใช้จัดการเกี่ยวกับเนื้อหาในอีเมลล์โดย Java mail ได้นำเอา RFC 822 และ MIME Internet Message Standard ซึ่ง Mime Message คลาสนั้นได้ทำการสืบทอดจากคลาส Message ซึ่งไว้ใช้ในการสร้างอีเมลล์ที่มีรูปแบบ MIME Message
- Store เป็นคลาสที่เกี่ยวข้องกับการอ้างถึงค่าแบบสของ Message ที่อยู่ใน Mail Server
- Folder เป็นคลาสที่เกี่ยวข้องกับการเก็บอีเมลล์ที่สามารถให้ยูสเซอร์สามารถสร้าง Folder ในการทำงานได้

- Transport เป็นคลาสที่เกี่ยวกับการจัดส่งตัวของอีเมลล์ โดยที่จะทำงานกับเมลล์โปรโตคอลที่กำหนด

โดยได้แสดงการใช้คลาสของ Java mail ในการอ่านและส่งอีเมลล์ผ่านโปรโตคอล IMAP4 และ SMTP ดังรูปที่ 2.6

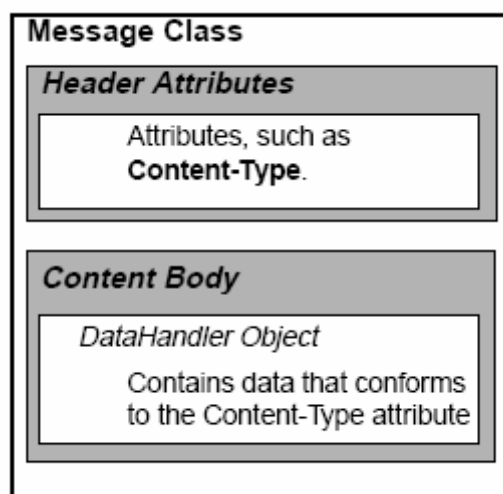


รูปที่ 2.16 แสดงการใช้คลาสใน Java Mail ในการอ่านและส่งอีเมลล์ผ่านโปรโตคอล IMAP และ SMTP

2.6.2 โครงสร้างของอีเมลล์ Message ใน Java Mail

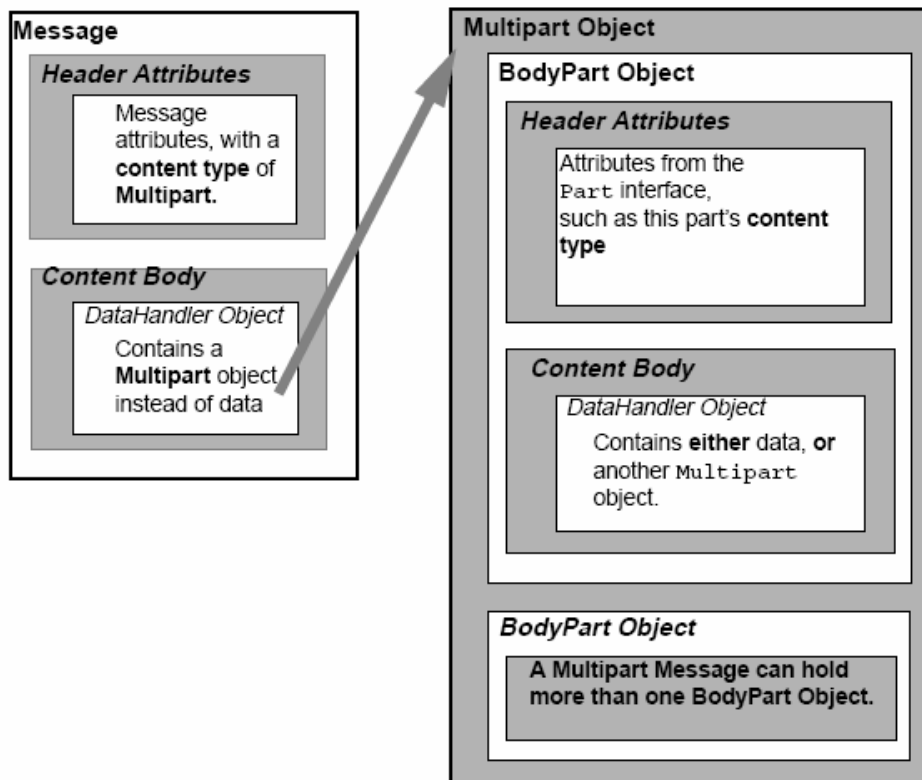
ในโครงสร้างของข้อมูลอีเมลล์นั้นจะประกอบไปด้วย โครงสร้างของ Message Content (Content Type) ซึ่งสามารถประกอบด้วยอีเมลล์ Single Content Object (ข้อมูลแบบเดียว) หรือ Multiple Content Object (ข้อมูลหลายแบบ) ซึ่ง Content Object นั้นจะถูกจัดการโดย DataHandler Object

Simple Message จะมี Single Content Object ซึ่งที่ถูกห่อหุ้มด้วย DataHandler Object แสดงในรูปที่ 2.17



รูปที่ 2.17 แสดงโครงสร้างของ Simple Message ใน Java Mail

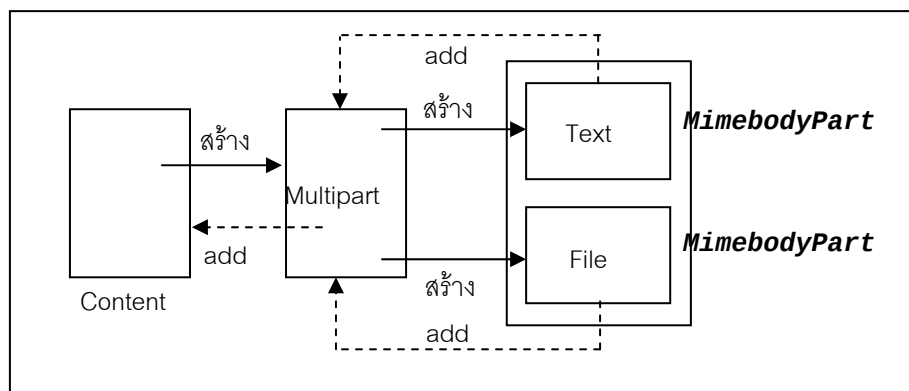
Multipart Message โดยที่ Message ประกอบด้วย Content Object หลายๆตัว โดยได้แสดงในรูปที่ 2.18 ซึ่งในรูปนี้ Handler Object ประกอบด้วย Multipart Object ซึ่งแทนที่ข้อมูลใน Simple Message โดย Multipart Object ประกอบด้วย Bodypart Object ซึ่งเป็นโครงสร้างของ Bodypart นั้นจะเหมือนกับ Simple Message ตัวหนึ่ง



รูปที่ 2.18 แสดงโครงสร้างของ Multipart Message ใน Java Mail

2.6.3 กระบวนการสร้างอีเมลโดยใช้ Java Mail

ในการเขียนโปรแกรมสำหรับการส่งอีเมลล์ด้วย Java Mail นี้จำเป็นต้องใช้ package *javax.mail* และ *javax.activation* โดยมีกระบวนการในการสร้างข้อความในอีเมลล์ รวมถึงการแนบไฟล์ต่างๆไปกับข้อความในอีเมลล์ ดังแสดงในรูปที่ 2.19



รูปที่ 2.19 แสดงการสร้างอีเมลล์ที่ทำการสร้างโดย Java Mail

กระบวนการสร้างอีเมลล์ เริ่มต้นด้วยการสร้างตัว Content ขึ้นมาก่อน โดยที่ Content จะทำการสร้าง Multipart เป็นลำดับต่อมา สำหรับ Multipart นี้ทำหน้าที่เหมือนกับส่วนที่บรรจุส่วนที่เป็นตัวเนื้อความ รวมถึงไฟล์ที่ต้องการแนบไปกับอีเมลล์และส่วนของ Multipart นี้จะทำหน้าที่สร้าง Mimebodypart เพื่อที่จะเขียนข้อความที่ต้องการส่งไป และ Multipart จะต้องสร้าง Mimebodypart อีกหนึ่งตัวเพื่อที่จะใส่ข้อมูลไฟล์ที่ต้องการแนบไปด้วย จากนั้นก็จะส่งกลับมาเป็นลำดับ จนถึงตัวของ Content ซึ่งจะทำให้ได้อีเมลล์ที่ต้องการส่งออกไป

2.7 การตรวจสอบการเคลื่อนไหวของวัตถุ

2.7.1 กระบวนการที่ใช้ในการตรวจสอบความเคลื่อนไหว

เนื่องจากข้อมูลภาพที่รับเข้าจะเกี่ยวข้องกับ RGB format แต่จะเป็นการลำบากต่อการคำนวณ และไม่จำเป็นที่จะวิเคราะห์แยกสีแต่ละสีเพื่อตรวจสอบการเคลื่อนไหว ดังนั้นอัลกอริทึมที่ใช้จึงสนใจเพียงแต่การแสดงออกของสีขาวและดำของข้อมูลที่ได้รับเข้าเท่านั้น(ใช้โหมดภาพขาวดำ) ซึ่งเป็นการตรวจสอบความเข้มสีที่เปลี่ยนแปลงไปของแต่ละพิกเซล

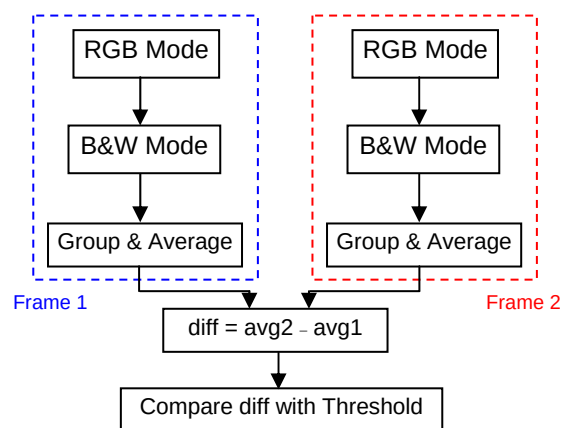
วิธีการตรวจสอบการเคลื่อนไหว[11]โดยการเปรียบเทียบแต่ละจุดภาพพิกเซลที่ตรงกันของภาพเฟรมก่อนหน้ากับเฟรมปัจจุบันอาจมีผลกระทบของสัญญาณรบกวนได้ ซึ่งสัญญาณรบกวนนี้จะมากหรือน้อยก็ขึ้นอยู่กับคุณภาพของข้อมูลวิดีโอที่รับเข้ามา วิธีการป้องกันหรือลดการรบกวนจากสัญญาณรบกวนทำได้โดยการจับกลุ่มจุดภาพพิกเซลใกล้เคียงกันไว้ด้วยกันจากนั้นคำนวณความเข้มเฉลี่ยของกลุ่มจุดนั้น(ค่าสีขาว,ดำ) และใช้ค่าเฉลี่ยนี้สำหรับการเปรียบเทียบกับค่าเฉลี่ยของเฟรมก่อนหน้า

การเปรียบเทียบจะประกอบด้วยการเปรียบเทียบค่าของสีเฉลี่ยของแต่ละอันกับค่าของสีเฉลี่ยนั้นในเฟรมก่อนหน้า ถ้ามีความแตกต่างกันระดับที่กำหนดแสดงว่าพื้นที่ที่สีเฉลี่ยนั้นครอบคลุมอยู่มีการเคลื่อนไหวที่ค่อนข้างมาก สำหรับขั้นตอนสุดท้ายต้องมีการตัดสินใจว่ามี

สีเหลี่ยมที่ช่องที่มีการเคลื่อนไหวจริงๆ ก่อนที่จะถูกพิจารณาว่ามีการเคลื่อนไหวในเฟรมวิดีโอ

ทั้งหมดดังแสดงใน

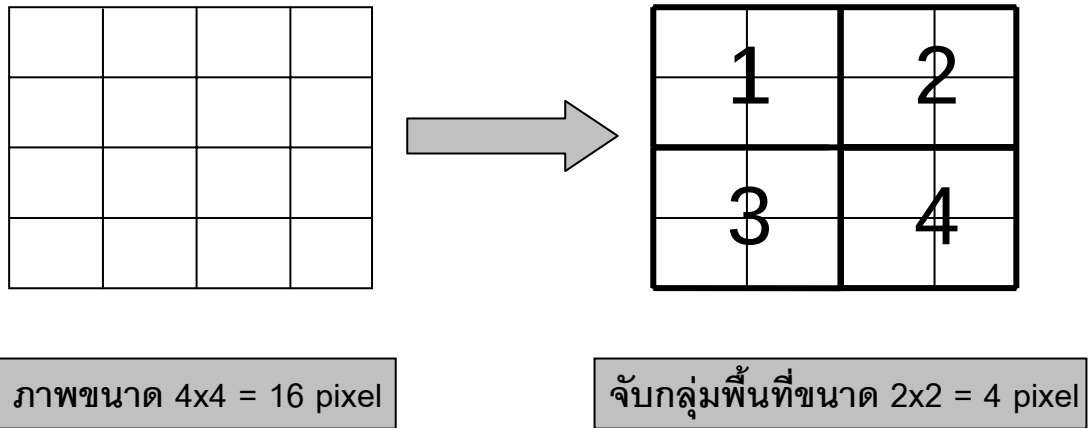
รูปที่ 2.20



รูปที่ 2.20 แสดงกระบวนการตรวจสอบการเคลื่อนไหว

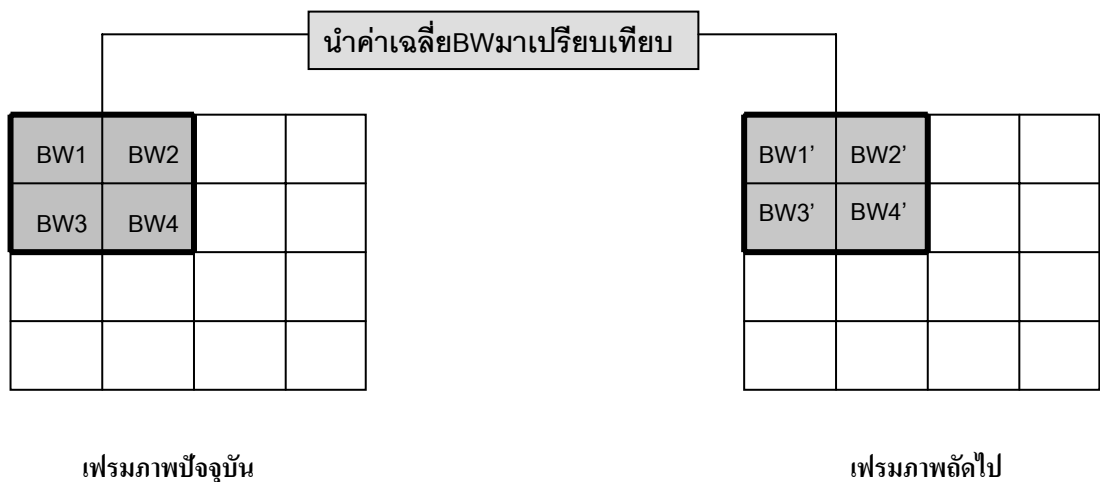
ซึ่งในรูปที่ 2.20 ขั้นตอนแรกจะทำการแปลงภาพจากโหมดภาพสี RGB และอยู่ในโหมดภาพขาวดำก่อน (B&W Mode) และนำกลุ่ม Pixel ที่ใกล้เคียงกันจับกลุ่มเข้าด้วยกันแล้วนำมาหาค่าเฉลี่ยสีขาวดำในแต่ละพื้นที่ที่จับกลุ่มเข้าด้วยกัน และนำค่าที่ได้นั้น นำมาเปรียบเทียบกับค่าที่ได้ในเฟรมภาพถัดไป จากนั้นนำค่าที่ได้จากการเปรียบเทียบนั้นนำมาทำการเปรียบเทียบกับค่า Threshold ที่กำหนด ถ้าค่าที่ได้มากกว่าที่ Threshold กำหนด จะถือว่าพื้นที่นั้นเกิดการเคลื่อนไหวของภาพเกิดขึ้น

ตัวอย่างวิธีแสดงการจับกลุ่มของภาพที่อยู่ใกล้เคียงกันเข้าด้วยกันเพื่อนำมาคำนวณสีความเข้มของสีในการตรวจสอบการเคลื่อนไหวของภาพแสดงดังรูปที่ 2.21



รูปที่ 2.21 แสดงตัวอย่างการจับกลุ่มภาพในการตรวจสอบการเคลื่อนไหว

ในตัวอย่างจากรูปที่ 2.21 แสดงขนาดภาพ 16 พิกเซล ทำการจับกลุ่มพื้นที่ขนาด 4 พิกเซล จะได้พื้นที่ขนาด 4 พิกเซล ออกมาเท่ากับ 4 พื้นที่ เมื่อนำมาจับกลุ่มภาพเข้าด้วยกันแล้วจะทำการหาค่าเฉลี่ยของสี่พื้นที่ทำการจับกลุ่มเข้าด้วยกัน และนำมาหาผลต่างของเฟรมภาพปัจจุบันกับเฟรมภาพถัดไปแสดงดังรูปที่ 2.22

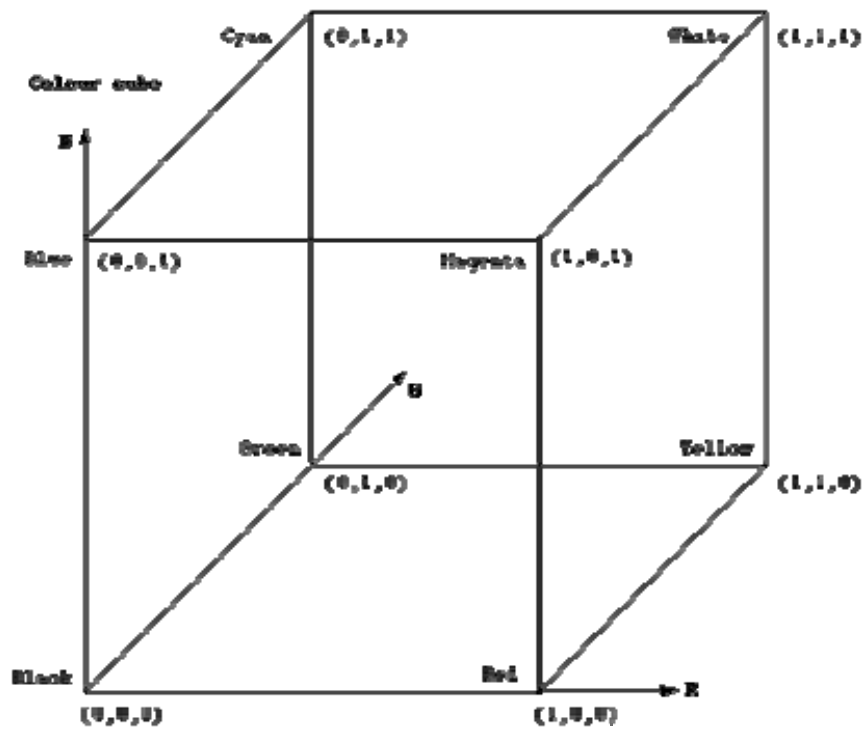


รูปที่ 2.22 แสดงตัวอย่างเปรียบเทียบกลุ่มภาพในการตรวจสอบการเคลื่อนไหว

ในรูปที่ 2.22 ได้นำพื้นที่ที่จับกลุ่มเข้าด้วยกันนั้นมาหาค่าเฉลี่ยของสี่พิกเซลในหน้านั้น โดยนำค่าความเข้มของสี BW1 BW2 BW3 BW4 มาหาค่าเฉลี่ย $(BW1+BW2+BW3+BW4)$ และนำผลที่ได้มาหาผลต่างความเข้มของสีในพิกเซลของสีเฟรมถัดไปในตำแหน่งเดียวกัน คือ $(BW1+BW2+BW3+BW4) - (BW1'-BW2'-BW3'-BW4')$ แล้วนำค่าที่ได้ไปเปรียบเทียบกับ ค่า Threshold ที่ได้ตั้งไว้ถ้ามากกว่าจะถือว่าพื้นที่นั้นมีการเคลื่อนไหวเกิดขึ้น

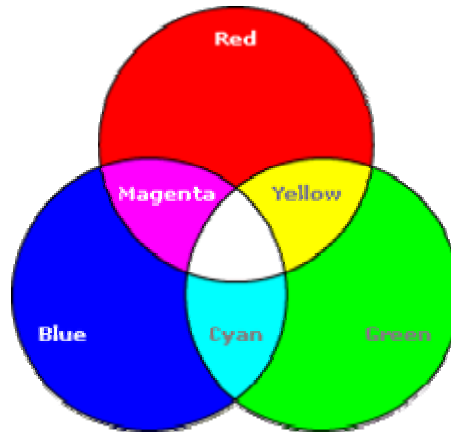
2.7.2 มาตรฐานของภาพสีในโหมด RGB

มาตรฐานของสีที่ใช้อยู่ในปัจจุบันมีอยู่หลายระบบด้วยกัน ขึ้นอยู่กับการใช้งานแต่ทุกมาตรฐานมีแนวคิดเดียวกันคือ การแทนจุดสีด้วยจุดที่อยู่ในสเปส 3 มิติ โดยจะมีแกนอ้างอิงสำหรับจุดสีนั้นในสเปส 3 มิติ ซึ่งแต่ละแกนจะมีความเป็นอิสระต่อกัน เช่น ในระบบ RGB จะมีแกนสี คือ แกนสีแดง เขียว และ น้ำเงิน แสดงอยู่ในรูปที่ 2.23



รูปที่ 2.23 แสดงสเปซสามมิติของโหมดสี RGB

ระบบสี RGB เป็นระบบสีที่เกิดจากการรวมกันของแสงสีแดง เขียวและน้ำเงิน โดยมีการรวมกันแบบAdditiveแสดงเป็นโมเดลดังรูปที่2.24 ซึ่งโดยปกติจะนำไปใช้ในจอภาพแบบ CRT (Cathode ray tube) ในการใช้งานระบบสีRGB ยังมีการสร้างมาตรฐานที่แตกต่างกันออกไปที่นิยมใช้งานได้แก่ RGB(CIE) เป็นระบบสีที่พัฒนาขึ้นโดยCIE (Commission International l'Eclairage) ซึ่งอ้างอิงสีด้วยสีแดงที่ 700nm สีเขียวเท่ากับ 546.1nm และสีน้ำเงิน 435.8nm และ RGB(NTSC) เป็นระบบที่พัฒนาโดย NTSC (National Television System Committee) เพื่อใช้สำหรับการแสดงภาพของจอภาพแบบ CRT



รูปที่ 2.24 แสดงโมเดลของโหมดสี RGB

2.7.3 การเปลี่ยนแปลงภาพจากภาพสีเป็นภาพขาวดำ

การเปลี่ยนแปลงภาพสีเป็นภาพขาวดำนั้น มีหลายเทคนิคในการเปลี่ยนแปลงภาพสี กลายเป็นภาพขาวดำหลายวิธีการคำนวณโดยได้แสดงการคำนวณไว้ 5 วิธีดังนี้โดยใช้ตัวอย่างภาพสีดังรูปที่ 2.25



รูปที่ 2.25 แสดงตัวอย่างของภาพสี

วิธีที่ 1 ใช้การเปลี่ยนแปลงภาพสีในการคำนวณค่าจาก สเตป 3 มิติ Color Cube ซึ่งเป็นวิธีที่ใช้มากที่สุด โดยมีสูตรในการหาคือ $\text{Sqrt}(\text{red} \times \text{red} + \text{green} \times \text{green} + \text{blue} \times \text{blue})$ คือค่าที่นำมาเทียบใน โหมดภาพขาวและดำ (Grey Mode) จะได้การแปลงภาพสีจากรูปที่ 2.25 เป็นภาพโหมดขาวและดำ ในรูปที่ 2.26



รูปที่ 2.26 แสดงตัวอย่างของภาพขาวดำวิธีที่ 1

วิธีที่ 2 ใช้การเฉลี่ยค่าสีในโหมด RGB โดยใช้สูตรเท่ากับ $(red+green+blue)/3$ คือค่าที่นำมาเปรียบเทียบกับโหมดขาวและดำ(Grey Mode) จะได้การแปลงภาพสีจากรูปที่ 2.25 เป็นภาพโหมดขาวและดำในรูปที่ 2.27



รูปที่ 2.27 แสดงตัวอย่างของภาพขาวดำวิธีที่ 2

วิธีที่ 3 ใช้การเฉลี่ยโดยคือน้ำหนักของสีในโหมด RGB ซึ่งมีสูตรการคำนวณที่เป็นที่นิยมนั่นคือ $(3 * red + 4 * blue + 2 * green) / 9$ คือค่าที่นำมาเทียบในโหมดขาวและดำ(Grey Mode) จะได้การแปลงภาพสีจากรูปที่ 2.25 เป็นภาพโหมดขาวและดำในรูปที่ 2.28



รูปที่ 2.28 แสดงตัวอย่างของภาพขาวดำวิธีที่ 3

วิธีที่ 4 ใช้ในระบบ NTSC และ PAL โดยใช้การสูตรการคำนวณคือ $(0.299 * red + 0.587 * green + 0.114 * blue)$ คือค่าที่นำมาเทียบในโหมดขาวและดำ(Grey Mode) จะได้การแปลงภาพสีจากรูปที่ 2.25 เป็นภาพโหมดขาวและดำในรูปที่ 2.29



รูปที่ 2.28 แสดงตัวอย่างของภาพขาวดำวิธีที่ 4